GLA UNIVERSITY, GREATER NOIDA

Greater Noida – 201310, Uttar Pradesh, India

AI Operations Assistant for Smart Inventory & Billing

SOFTWARE REQUIREMENT SPECIFICATION

Version 1.0

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING - AI

Submitted By

RAM GANESH KINTALI

Roll No.: 22515500047 | Section: NE

Under the Supervision of

Mr. Palash Singh

GLA University, Greater Noida

Reviewed By

Mr. Gautam Mukherjee

GLA University, Greater Noida

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING / APPLICATIONS GLA UNIVERSITY, GREATER NOIDA – 201310

SEPTEMBER, 2026

Revision History

Version	Date	Change	Reason
1.0	04-09-2026	Initial version	First submission

1. Introduction

1.1 Purpose

This document specifies the functional, non-functional, data, architecture, acceptance, and release requirements for version 1.0 of the AI Operations Assistant for Smart Inventory & Billing. It defines the baseline against which the implementation and test cases will be verified.

1.2 Scope

The system provides authenticated shop-owner and store-manager workflows for product management, billing, stock adjustments, expiry monitoring, sales analytics, seven-day demand prediction and reorder recommendations. The system does not manage suppliers, place automatic supplier orders, support multiple shops, or provide a generative-AI business chatbot.

1.3 Definitions

Term	Definition
Reorder window	The seven-day period for which the system predicts product demand and calculates a reorder quantity.
Stock discrepancy	The difference recorded when a sale quantity exceeds the system-recorded stock and the sale is allowed to proceed.
Safety stock	An additional quantity added to predicted seven-day demand to reduce the risk of stock shortage.
Demand prediction	The predicted number of units of a product expected to be sold during the next seven days.

2. Overall Description

2.1 User Classes

User class	Who they are	What they may do
Shop owner	The authenticated owner of one Kirana shop.	Manage products; view inventory, expiry, sales, analytics and predictions; refresh predictions; make stock adjustments.
Store manager	The authenticated manager who operates daily store workflows.	Create bills; view products and stock; record stock adjustments; view expiry information and demand predictions.

2.2 Assumptions and Dependencies

- The operational database is PostgreSQL; if it is unavailable, the application shall show a database error and shall not claim the write succeeded.
- The ML service uses Python with Pandas, NumPy and scikit-learn; if prediction is unavailable, the application shall use the Level 0 baseline estimate.
- Historical demand is derived from the UCI Online Retail transaction dataset; application-only fields such as current stock, expiry date, minimum stock and safety stock are maintained in PostgreSQL.

2.3 Constraints

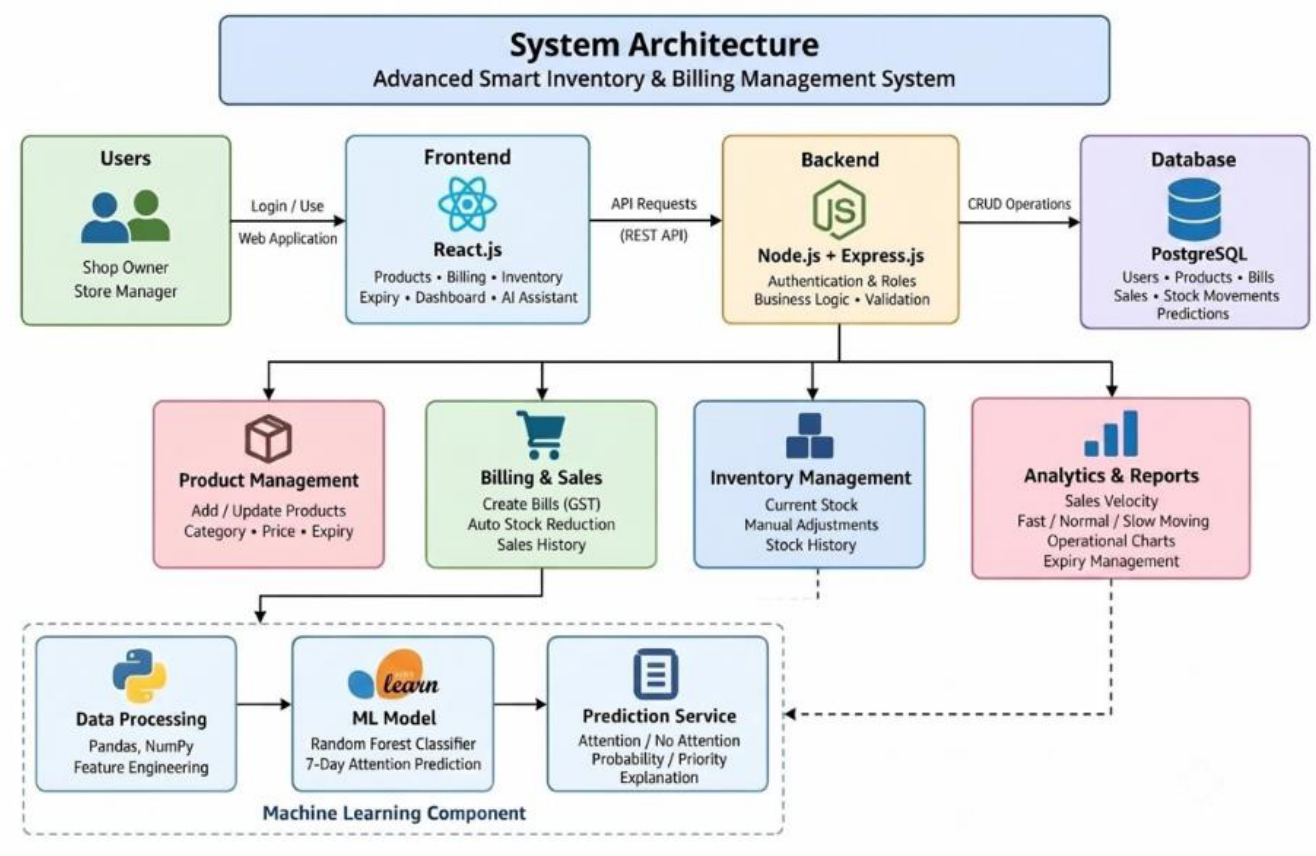
C1 The UCI Online Retail dataset is transaction data and does not contain current stock, expiry dates, minimum-stock levels, or supplier information; these values must therefore be maintained by the application database.

C2 The release is limited to one Kirana shop, packaged/branded FMCG products, and approximately 90 development hours, with approximately 20 hours allocated to modelling.

3. Data Source and Baseline

Item	Value	Required now?
Dataset name	UCI Online Retail Dataset	Yes
Source and URL	UCI Machine Learning Repository — https://archive.ics.uci.edu/dataset/352/online+retail	Yes
Licence / terms of use	UCI Machine Learning Repository dataset terms of use; verify the current UCI licence/usage terms before redistribution.	Yes
Number of rows and columns	541,909 transactions and 8 columns; 4,070 products, 25,900 invoices, 4,372 customers, 38 countries.	Yes
Prediction target	Future seven-day product demand: total units expected to be sold for each product over the next seven days.	Yes
Task type	Predicting a number (regression).	Yes
Class balance	N/A — regression task.	Yes
Primary evaluation metric	MAE (Mean Absolute Error).	Yes
Why that metric	MAE measures the average absolute difference in units, matching the shop owner's practical over-ordering or under-ordering error.	Yes
Level 0 baseline and its score	Level 0 predicts the overall training-set mean for every test observation. Score: not computed in the supplied project context; it must be measured on the final train/test split before submission.	Yes
Known data quality problems	Cancelled invoices; negative/zero quantities; zero/negative prices; duplicate transactions; CustomerID missing in some records. CustomerID is not required for product-demand prediction.	Yes
Level 1 baseline and its score	A simple product-based/shallow baseline will be evaluated using the final SRS split; score not yet available in the supplied project context.	Later
Target for this release	Random Forest Regressor shall demonstrate lower MAE than the Level 1 baseline on the defined test set.	Later

4. System Architecture



Component	Responsibility
React frontend	Provides the user interface for dashboard, billing, inventory, expiry, analytics, and prediction screens.Allows users to view and interact with the system features.
Node.js + Express API	Handles authentication, billing, inventory operations, product management, and API requests.Connects the frontend with the database and Python ML service.
PostgreSQL	Stores users, products, bills, bill items, sales transactions, and stock movements. Also stores prediction records required by the system.
Python ML pipeline	Cleans historical transaction data and prepares features such as lag, rolling-average, calendar, price, and product features. Provides the processed features to the machine-learning model for demand prediction.
Random Forest Regressor	Uses supervised-learning features to predict seven-day product demand. The prediction is used to support the system's reorder recommendation.

5. Functional Requirements

5.1 Operations, Inventory and Billing Requirements

FR-1.1 The system shall authenticate a registered shop owner or store manager before allowing access to protected application functions.
FR-1.2 The system shall allow an authenticated user to create, edit and view product records containing product name, product code, unit price and stock information.
FR-1.3 The system shall create a bill containing at least one product and a positive quantity and shall save the bill and its bill items in the database.
FR-1.4 The system shall reduce recorded stock by the quantity sold when a bill is saved successfully.
FR-1.5 The system shall allow an authenticated user to record a stock adjustment with product, quantity change and reason.
FR-1.6 The system shall identify products whose recorded stock is at or below their configured minimum-stock level.
FR-1.7 The system shall display products with expiry dates within the configured approaching-expiry period in a separate expiry section.
FR-1.8 The system shall display sales history grouped by product and date for the available recorded sales transactions.
FR-1.9 The system shall, when a bill quantity exceeds recorded stock, display an insufficient-stock warning before completion of the sale.

FR-1.10 The system shall require user acknowledgement before allowing a sale whose quantity exceeds recorded stock.
FR-1.11 The system shall allow the user to complete an acknowledged insufficient-stock sale without reducing recorded stock below zero.
FR-1.12 The system shall record a stock discrepancy when an acknowledged insufficient-stock sale is completed.

5.2 Model-backed Requirements None

FR-2.1 The system shall accept a valid product and prediction date request and shall reject a request for an unknown product or invalid prediction date with an error message.
FR-2.2 The system shall return a predicted seven-day demand quantity and a reorder quantity for a valid product prediction request.
FR-2.3 The system shall present the predicted seven-day demand, current stock, safety stock and recommended reorder quantity together in the prediction view.
FR-2.4 The system shall, when the Random Forest prediction service is unavailable or returns no usable prediction, display the Level 0 baseline quantity labelled as an estimate instead of an empty result.
FR-2.5 The system shall allow the shop owner to override the recommended reorder quantity before using it for a stock decision.

6. Non-Functional Requirements

NFR-1 The system shall return a prediction response for a shop with up to 500 products within 4 seconds at the 95th percentile under the project test environment.
NFR-2 The system shall protect passwords using salted password hashing and shall require authenticated access to protected API endpoints.
NFR-3 The system shall record each prediction with product identifier, prediction date, predicted quantity, model version and evaluation/fallback status.
NFR-4 The system shall preserve stock and billing writes transactionally in PostgreSQL so that a failed bill save shall not partially update recorded stock.

7. Build Plan

7.1 Minimum Viable Product

FR / NFR ID	Requirement (short)	Why this one is in the MVP
FR-1.2	Product management	The downstream billing and inventory flows require a stable product catalogue.
FR-1.3	Bill creation and storage	Billing is the primary operational event that creates sales history for later analysis.
FR-1.4	Automatic stock reduction	Without stock reduction, inventory and reorder calculations cannot reflect actual recorded sales.
FR-1.5	Stock adjustment	Manual correction is required because physical stock can differ from recorded stock.

FR / NFR ID	Requirement (short)	Why this one is in the MVP
FR-2.4	Prediction fallback	A working application must remain demonstrable even if the ML service is unavailable.
NFR-1	Prediction response time	A measurable response limit is needed to make the prediction screen testable during the demo.

NFR-2	Password hashing and authenticated endpoints	The application contains user accounts and operational shop data, so access control is required in the MVP.

7.2 Build Order

Stage	Deliverable	Date
Development Sprint I	Planning / Researching for Dataset	05-08-2026 to 05-09-2026
Development Sprint II	50% functional application	06-09-2026 to 17-10-206
Development Sprint III	75–80% functional application	18-10-2026 to 7-11-2026
Testing & Validation	Test report and bug log	8-11-2026 to 18-11-2026
Deployment & Documentation	Deployed application, user manual, technical documentation	19-11-2026 to 30-11-2026
Final Presentation	Final report, presentation and viva	05-12-2026 to 10-12-2026

Stage	What will be working by then	FR / NFR IDs	How I will know it is done
Sprint I 5 aug to 5 sep	Authenticated prototype with product records, billing persistence, stock reduction, stock adjustment and baseline fallback.	FR-1.2, FR-1.3, FR-1.4, FR-1.5, FR-2.4, NFR-1, NFR-2	Demonstrate login, add product, create bill, show stock reduction, make stock adjustment, and display fallback estimate.
Sprint II 6 sep to 17 Oct	Inventory alerts, expiry section, sales history and core analytics.	FR-1.1, FR-1.6, FR-1.7, FR-1.8, FR-1.9, FR-1.10, FR-1.11, FR-1.12, NFR-4	Demonstrate low-stock and expiry views, sales history, and an insufficient-stock sale with discrepancy recording.
Sprint III 18 oct to 7 Nov	Random Forest prediction, prediction display, reorder recommendation and owner override.	FR-2.1, FR-2.2, FR-2.3, FR-2.5, NFR-3	Demonstrate a product prediction, reorder calculation, recorded model version, and manual override.
Testing 8 Nov to 18 Nov	MVP and model validation test report and bug log.	All FR-1.1–FR-1.12, FR-2.1–FR-2.5, NFR-1–NFR-4	Run acceptance tests, failure-path tests, response-time checks, security checks and MAE evaluation.
Deployment 19 Nov to 30 Nov	Deployed application plus user manual and technical documentation.	All release requirements	Open deployed application, authenticate, demonstrate core workflow, and show documentation.

8. Data Requirements

8.1 Entities

Entity: Bill		
Field	**Type**	**Constraint / Notes**
id	UUID	Primary key.
user_id	UUID	Not null; references authenticated user.
bill_date	TIMESTAMP	Not null; recorded when the bill is created.
total_amount	NUMERIC(12, 2)	Not null; must be zero or greater.
created_at	TIMESTAMP	Not null; generated by the database/application.

Entity: Users		
Field	**Type**	**Constraint / Notes**
id	UUID	Primary key; identifies the authenticated user.
role	TEXT	Not null; shop owner or store manager.

Entity: Products		
Field	**Type**	**Constraint / Notes**
id	UUID	Primary key.
product_code	TEXT	Not null; unique product identifier/code.
product_name	TEXT	Not null; product name.
unit_price	NUMERIC(12, 2)	Not null; unit selling price.
stock	INTEGER	Not null; recorded stock must not become negative.
minimum_stock	INTEGER	Not null; threshold for low-stock identification.
expiry_date	DATE	Optional; used by the approaching-expiry view.
safety_stock	INTEGER	Not null; additional stock used in reorder calculation.

Entity: Bill Items		
Field	**Type**	**Constraint / Notes**
id	UUID	Primary key.
bill_id	UUID	Not null; references Bill.
product_id	UUID	Not null; references Product.
quantity	INTEGER	Not null; must be positive.
unit_price	NUMERIC(12, 2)	Not null; price recorded at billing time.

Entity: Sales Transactions		
Field	**Type**	**Constraint / Notes**
id	UUID	Primary key.
product_id	UUID	Not null; references Product.
bill_id	UUID	Not null; references Bill.
sale_date	TIMESTAMP	Not null; recorded date/time of sale.
quantity	INTEGER	Not null; units sold used to derive historical demand.

Entity: Stock Movements		
Field	**Type**	**Constraint / Notes**
id	UUID	Primary key.
product_id	UUID	Not null; references Product.
quantity_change	INTEGER	Stock increase or decrease.
reason	TEXT	Not null; reason for the stock adjustment.
created_at	TIMESTAMP	Not null; time the stock movement was recorded.

Entity: Prediction Records		
Field	**Type**	**Constraint / Notes**
id	UUID	Primary key.
product_id	UUID	Not null; references Product.
prediction_date	DATE	Not null; date of the prediction request.
predicted_quantity	NUMERIC	Not null; predicted seven-day demand quantity.
model_version	TEXT	Version of the model used for the prediction.
fallback_status	TEXT	Records whether model prediction or Level 0 fallback was used.

9. Design Decisions and Alternatives Considered

#	Decision	Alternative rejected	Reason
D1	Use UCI Online Retail transaction data as historical demand source	Use a synthetic inventory dataset as the primary ML source	The supplied project dataset has 541,909 transactions and 8 columns and is explicitly selected for the project; application-only inventory fields are maintained separately.
D2	Use Random Forest Regressor with lag and rolling features	Use ARIMA or another dedicated time-series forecasting algorithm	The project is based on supervised ML techniques and has limited modelling time; Random Forest handles nonlinear relationships and is easier to demonstrate with derived features.
D3	Allow acknowledged insufficient-stock sales while keeping recorded stock non-negative	Block every sale when recorded stock is insufficient	Recorded stock may differ from physical stock in a Kirana shop; blocking the sale would prevent the system from representing the real-world discrepancy and correction workflow.

10. Acceptance Criteria

AC-1 A user logs in, creates a product, creates a bill for that product, and submits the bill; the product's recorded stock decreases by the billed quantity and the bill is visible in sales history.
AC-2 A user records a stock adjustment for a product; the inventory view immediately reflects the adjusted recorded stock and the adjustment is stored in stock movements.
AC-3 A user attempts to sell more units than recorded stock; the system displays an insufficient-stock warning, requires acknowledgement, permits Sell Anyway, keeps recorded stock non-negative, and records a stock discrepancy.
AC-4 The shop owner requests a prediction for a valid product; the system displays seven-day demand, current stock, safety stock and recommended reorder quantity, and permits the owner to override the recommendation.

11. Out of Scope for This Release

Supplier management and automatic supplier ordering are out of scope because supplier records, supplier lead times and order workflows are not provided by the selected dataset and are not included in v1.0.
Multiple-shop support is out of scope because v1.0 is deliberately limited to one Kirana shop and a single operational database context.
Generative-AI chatbot/business conversation is out of scope for this release because the project's AI component is the seven-day demand prediction and reorder recommendation workflow.